

Towards desktop personal travel agents

D T Ndumu, J C Collis and H S Nwana

The growth of the World Wide Web has invested personal computer users with direct access to a wealth of information sources and services — for example, one can now check stock market share prices, book flight tickets or even do grocery shopping over the Internet. However, while the Internet enables direct access to various information sources and services, effectively integrating these services to provide a total solution to a complex task, such as arranging a transatlantic trip itinerary, remains a challenging task that requires significant human intervention. This paper discusses some of the challenges involved in providing computer-based integrated personalised travel services through a medium such as the Internet. Further, a software agent solution to this problem is presented, and an agent-based travel assistant demonstrator described, which was developed using the ZEUS collaborative agent building tool-kit.

1. Introduction

This paper describes a personal travel assistant (PTA) system that arranges trip itineraries on behalf of its users by integrating distributed travel services. Our objective is to provide personalised journey information and assistance to potential travellers. Until recently, such a service would only have been available from travel agencies, but the increasing popularity of the Internet as a medium for doing business has led to a variety of travel services becoming available on-line. Hence the possibility of arranging an entire trip itinerary from a traveller's personal computer, using information gathered from the World Wide Web (WWW), and the traveller's own personal preferences.

Consider, for example, the following scenario — a traveller wishes to arrange a trip from a town outside London to a city on the eastern coast of the USA. Ideally these requirements could be submitted once to an on-line travel assistant, that could use its knowledge of the travel industry to plan an appropriate itinerary. This hypothetical travel assistant would consult the global flight schedules to determine which airlines serve the chosen destination, and the airports from which they depart. Information from the national UK rail timetables could then be used to determine what time the traveller would need to leave home to reach the airport with adequate time to check in. The assistant could also arrange hotel accommodation for the duration of the traveller's stay, and provide a list of local attractions that might interest them. When the required services are booked, the assistant could then be instructed to monitor news sources for any events that might conceivably delay or jeopardise the journey, such as adverse weather or strikes by air-traffic controllers.

To implement such a hypothetical personal travel assistant, three significant issues need to be considered.

- Personalisation

The assistant needs to learn about its user's interests and preferences in order to personalise the services it books. Such learning could be performed by adaptive profiling [1], whereby machine learning techniques are used to learn users' preferences by observing their interactions with different on-line information sources. Information from a user's profile will influence how the assistant ranks and selects equivalent travel services without needing the user to arbitrate.

- Pre-trip planning

The PTA will use its owner's profile and requirements, such as the destinations to be visited and the duration of the stay, to automatically locate, select and interact with the appropriate Internet information sources and services. It should construct potential itineraries personalised to the user's interests and preferences, and forward them to the user to choose the preferred journey details. Such a process typically involves some reasoning and planning, e.g. checking flight and hotel availability before choosing local transportation. Once the user is satisfied with the itinerary, the PTA can be authorised to make the appropriate travel and hotel bookings, which can be paid for electronically over the Internet using the user's credit card.

- Post-plan monitoring

Once a trip has been planned, the PTA should be able to use on-line information sources to monitor factors that might affect it. For instance, before the trip the PTA could report on current airport weather conditions and flight delays; and once the traveller has left for the airport, the PTA can monitor motorway traffic congestion reports and telephone them to the traveller's hands-free mobile phone.

None of these issues seems beyond present software technology, so why does such an undoubtedly useful personalised travel service not yet exist? One reason is the sheer difficulty of finding the right information. At present the information necessary to comprehensively plan a trip using different transport and accommodation services tends to be distributed across the Internet on the web sites of individual companies. For instance it is possible to find rail timetables¹, flight schedules², and hotel vacancy information³, on the Web. However as the quantity of information available on-line continues to grow, so does the problem of finding and integrating the available information to satisfy a traveller's personal requirements and constraints.

In other words, while the WWW is an ideal medium for disseminating information to individual users, it is very difficult to find and integrate information from multiple web sites. This is not surprising given that the WWW was developed to allow people to interact with remote computers. When computers need to communicate with each other, they do so directly, which raises the question: 'Why is the PC on your desk unable to contact the appropriate travel companies on your behalf and arrange a journey?'

This paper addresses the question by discussing some of the challenges involved in providing integrated personalised travel services through media such as the Internet. In section 2, the technical challenges that need to be resolved to make a PTA application feasible are described. Section 3 introduces collaborative agent technology, a potential solution that involves a society of distributed co-operating software entities. Section 4 describes an agent-based demonstrator of the travel assistant implemented using the ZEUS agent-building tool-kit [2], (albeit without personalisation and post-plan monitoring capabilities). Section 5 places the work in context by reviewing related work on using agents to realise travel services. Section 6 concludes the paper by summarising its main points and outlining areas for future work.

¹ <http://www.railtrack.co.uk/travel/>

² http://www.flifo.com/cgi-programs/hpage.cgi?la_ae+co_uk+ht_/ae/uk/

³ http://dps1.travelocity.com/!glogin.clt?tr_module=HT&Service=FORTEHOTELS

2. Challenges for the creators of a personal travel assistant

Currently there are several significant technical problems preventing the deployment of personal travel assistant software (of the type described in section 1). Most of these problems are rooted in the poor or non-existent interoperability between the many and diverse heterogeneous and distributed computing systems that would be involved in the provision of an integrated service. Some of the main problems are described in the following sections.

2.1 The information discovery problem

In the travel scenario outlined earlier, the PTA must obtain travel information such as timetables and seat availability from the databases of appropriate travel companies. Hence it needs some means of discovering what relevant resources exist, and where (on the Internet) they can be found. In other words, we need an equivalent of the 'Yahoo!'⁴ and 'Yell!'⁵ indices that provides 'Yellow Pages'TM-type service listings, rather than Web-site directories.

2.2 The communication problem

Suppose a solution to the information discovery problem exists, and the PTA is able to locate a flight reservation service on the Internet. In order to exploit this resource, the PTA must communicate with the reservation system; this raises three challenges. Firstly, both parties must adopt a common transport protocol that enables the request and results to be transmitted, as shown by the pipe in Fig 1. Fortunately, solutions to this problem abound, e.g. the TCP/IP protocol (transmission control protocol/Internet protocol), which facilitates the transfer of information across the Internet.

The next challenge is to devise a common communication language that both parties understand. Currently, the lingua franca of the Internet is the hypertext mark-up language (HTML), whose standard format enables browser software to interpret pages on the World Wide Web. However, HTML web pages were designed for presentation of information for human perusal, and so valuable information is typically intermingled with formatting instructions — necessitating 'wrapper induction' software programs [3], to parse the pages and extract the information from the formatting instructions. Clearly, in this travel scenario, a language geared towards direct machine-to-machine communication is preferable.

With the transport and structure of messages agreed, the final challenge is for the PTA and flight reservation system

⁴ <http://www.yahoo.co.uk>

⁵ <http://www.yell.co.uk>

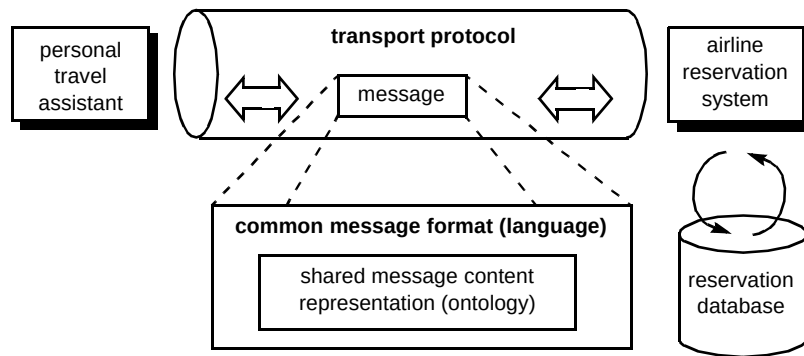


Fig 1 The three prerequisites for effective communication — for the PTA to make a flight reservation there must be some means of transmitting the request to the airline reservation system; for both parties to understand one another, their messages must be in a common language that is grounded in a shared ontology; once a message is received, the reservation system will use its knowledge of the ontology to translate the request into the terms used within its reservation database.

to agree on the list of terms to be used in the content of messages, and the meanings of these terms. This obstacle is known as the ontology problem, and is perhaps the main problem preventing widespread interoperability of heterogeneous, distributed co-operating systems⁶; we expand on this problem next.

2.3 The ontology problem

The ontology problem can be illustrated by returning to the scenario depicted in Fig 1. For the PTA and flight reservation system to interact, both systems must agree on and then share a common definition of travel-related concepts. The ontology (or concept definitions) specifies the terms each party must understand and use during communication, e.g. definition of a journey leg, identities of airlines, types of fares, etc. Creating an ontology involves explicitly defining every concept to be represented; take journey legs for example. What are the attributes of a leg, and what does each attribute mean pragmatically? Also what are the constraints on valid attribute values, and how are the attributes of one concept related to those of another?

2.4 The legacy software integration problem

Legacy systems are large pieces of software, based on older technologies, and generally not designed for interoperability with other systems. The reservation database of Fig 1 is an example of a legacy system. The legacy system problem involves devising some mechanism that enables legacy systems to communicate with external systems like the PTA. As Fig 1 suggests, the solution is likely to involve a proxy, e.g. the airline reservation system, capable of translating requests made using the shared ontology into queries in the internal language of the legacy system, and then translating the results received from the legacy system back into the shared ontology.

2.5 The reasoning and co-ordination problem

Now suppose that solutions to the aforementioned problems exist, and the PTA is able to successfully retrieve timetable information from various travel companies. The PTA must then plan a journey, which is likely to consist of several legs, from the traveller's home to their desired destination. This process necessitates reasoning, planning and constraint satisfaction capabilities in order to deal with timing and other dependencies in the travel itinerary, while also attempting to satisfy the traveller's preferences. In essence, the PTA needs to co-ordinate the services of the different service providers so that as a whole, the service providers behave coherently in their attempt to provision an integrated service. For instance, the PTA should be able to reason that the flight and hotel accommodation should be booked prior to considering local transportation, and ensure that a taxi is booked to coincide with the scheduled train arrival time.

2.6 The monitoring problem

The problems of information discovery, communication, ontology and planning manifest themselves again during post-plan monitoring. In order to react proactively to external events that might jeopardise successful execution of a planned trip, the PTA needs to discover relevant news sources, and periodically monitor them for news on events that might affect the trip. Next, it needs to evaluate whether and how any such events might jeopardise the trip, and then determine corrective actions (which might involve re-planning) to counter their effects. The Internet now has many such information sources, e.g. weather reports⁷, reports on road traffic conditions⁸ and news on airport strikes⁹. As such news sources present information in natural language, the key challenge is to build ontologies rich enough to represent the underlying concepts involved, so allowing agents to reason with the information obtained.

⁶ For example, progress towards large-scale electronic commerce is plagued by the lack of interoperable product catalogues [4].

⁷ <http://www.meto.govt.uk/sec3/sec3.html>

⁸ <http://www.rac.co.uk/html/jou/fjourney2.htm>

⁹ <http://www.abcnews.com>

Notwithstanding the above problems, some progress can be made towards realising the dream of having software systems that could provide integrated personalised travel services. The next section introduces collaborative software agents, an emerging technology that addresses some of these problems.

3. Collaborative software agents — a potential solution

The field of software agents is a relatively new technology that seems to offer promising solutions to most of the problems highlighted in the previous section. While there is a large body of ongoing research and development work under the banner of software agents, there is still no consensus definition of what is or is not an agent [5, 6]. In this paper, the definition of an agent proposed by Jennings and Wooldridge is adopted [7] as ‘a self-contained program capable of controlling its own decision-making and acting based on its perception of its environment, in pursuit of one or more objectives’. In other words, an agent typically works for and/or on behalf of a user, and acts to support the user in achieving their objectives. Jennings and Wooldridge specify four main attributes that determine agent-hood:

- autonomy — the ability to function largely independent of human intervention,
- social ability — the ability to interact ‘intelligently’ and constructively with other agents or humans,
- responsiveness — the ability to perceive the environment and respond in a timely fashion to events occurring in it,
- pro-activity — the ability to take the initiative whenever the situation demands.

Nwana and Ndumu [5] present a complementary view of agent-hood that also includes learning ability, i.e. the ability to improve performance over time.

Collaborative agents are a class of agent system that emphasises autonomy and social ability, wherein a number of different autonomous agents collaborate to achieve their objectives. The collaboration is necessary because expertise and/or resources may be distributed across the community of agents, and therefore no one agent is, on its own, capable of performing the required task. In order to ensure coherent execution of their joint endeavour, the agents need to plan and/or negotiate the terms and conditions of each individual’s contribution. Such negotiation typically involves the exchange of high-level messages concerning for example, requests for information or specification of contractual obligations. Collaborative agent technology has been applied to a number of applications requiring co-operation between disparate autonomous systems, e.g. workflow management [8] and business process

management [9], telecommunications network management [10], air traffic control [11] and 3-D scientific data visualisation [12].

The parallels between collaborative agent technology and the PTA problem should be apparent, indeed to the extent that FIPA [13], an agent standardisation body, has specified the PTA problem as one of the test application areas of its emerging standards for agent interoperability. In this hypothetical scenario, the travel assistant displays the characteristic agent attributes of:

- autonomy — largely by functioning independently of user assistance,
- social ability — by communicating and negotiating with remote independently autonomous systems and service providers such as flight and hotel reservation systems,
- responsiveness — by monitoring external information sources for news of events that might adversely affect planned trips,
- proactivity — by spontaneously initiating re-planning to counter the effects of external events that are likely to affect planned trips.

The remainder of this section will consider the techniques that collaborative agent technology has developed (or adapted from classical AI and software engineering) in order to deal with the problems outlined in section 2.

3.1 Tackling the information discovery problem

Given the frequency that Internet addresses change, it is wise to avoid hard-coding the addresses of resources into agents. Agent systems therefore have to solve the information discovery problem by delegating responsibility for maintaining indices of available resources to two specialised types of agent that serve whole agent communities. To find a particular resource an agent typically asks a facilitator, which is a ‘yellow pages’ service like the ‘Yahoo!’ directory. Once it has identified an information provider, an agent obtains its network location by querying an agent name server. This provides a ‘white pages’ service, mapping machine names to their physical location on a network just as Internet domain name servers translate Internet addresses to Internet protocol (IP) numbers.

3.2 Tackling the communication problem

While protocols like TCP/IP can facilitate information transfer between machines, no standard language exists to express the structure and content of messages. This problem requires the communicating parties to agree on what

instructions, assertions, requests, etc, will be supported, and what syntax is used. A number of inter-agent communication languages have been proposed, notably KQML (knowledge query and manipulation language) [14] and FIPA ACL (agent communication language) [13]. These languages are based on speech act theory performatives [15], wherein the speaker's intent of the effects of a message on the hearer is communicated by specifying the type of the message, e.g. ask, tell, or achieve message types. Furthermore, most of the languages do not specify a syntax or semantics of the contents of the messages, with the rationale being that different application domains may require different content languages. Nonetheless, a number of general purpose content languages have been developed, e.g. KIF (knowledge interchange format) [16], typically used with KQML, and FIPA SL [12] the preferred content language for use with the FIPA ACL.

3.3 *Tackling the ontology problem*

While some general-purpose ontologies have been proposed for use in inter-agent communication, like Cyc, [17], the current trend is towards the provision of editors for creating domain-specific ontologies and converters for translating between ontologies (see, for example, Gruber [18]), the argument being that, firstly, most general-purpose ontologies are unlikely to include the intricacies of all possible domains, and secondly that they are likely to be bloated and unnecessarily complex for most applications.

3.4 *Tackling the legacy system problem*

Genesereth and Ketchpel [19] discuss the problem of integrating legacy software with agent systems, and suggest three possible solutions to the problem:

- firstly, rewriting the software — a costly approach,
- secondly, through use of a separate piece of software called a transducer that acts as an interpreter between the agent communication language and the native protocol of the legacy system,
- lastly, the wrapper technique where the legacy program is augmented with code that enables it to communicate using the inter-agent language.

3.5 *Tackling the reasoning and co-ordination problem*

The reasoning problem is handled in most collaborative agent systems using AI techniques such as rule-based inference, classical planning, various logic formalisms, and constraint satisfaction techniques. Co-ordination to ensure coherent functioning of the multi-agent society is also handled with a variety of approaches including organisational structuring, contracting, multi-agent planning and negotiation [20]. In organisational structuring the pre-defined structure of the society (i.e. the roles of the different

agents and their relationships with one another) is exploited for co-ordination. This is typified by client/server systems, but also includes systems where agents engage in activities according to their roles; thus co-ordination is implicitly performed by assigning a role to an agent.

Contracting, as a co-ordination mechanism, is typified by the classic contract net protocol [21], where a manager agent announces a contract, receives bids from other interested contractor agents, evaluates the bids and awards the contract to a winning contractor. The simplicity of this scheme makes it one of the most widely used co-ordination mechanisms with many variants in the literature. In multi-agent planning, the agents utilise classical AI planning techniques to plan their activities, resolving any foreseen conflicts.

With negotiation, the agents engage in dialogue, exchanging proposals, evaluating other agents' proposals and then modifying their own proposals until a state is reached when all agents are satisfied with the set of proposals. Typical negotiation mechanisms are based on game theory, some form of planning, or on human-inspired negotiations. Nwana et al [20] present a gentle introduction to the literature on co-ordination of multi-agent systems.

3.6 *Tackling the monitoring problem*

Because of the dependence of monitoring on solving the problems of information discovery, communication, ontology, reasoning, and even the legacy software problem, no general-purpose total solution can be proposed to this situation. Thus, it is typically resolved in an application-specific manner using a combination of techniques. In general, however, solutions are based on software sensor mechanisms that react to changes in the sensed environment and notify the monitoring agent of these changes.

Having reviewed the agent-based solutions to the problems inherent in the PTA scenario, Fig 2 presents an agent-oriented solution to the PTA problem itself. As shown in Fig 2, the different participants in the PTA scenario are represented by software agents, with the agents communicating via a common inter-agent communication language and using a shared travel ontology. To arrange a complex trip, the travel assistant, imbued with adequate reasoning mechanisms, negotiates the different parameters of the trip with the relevant agents in the environment. The information discovery problem is handled through use of agent name servers and facilitators. The reservation agents in the scenario connect to their respective databases using any of the legacy software integration solutions presented earlier.

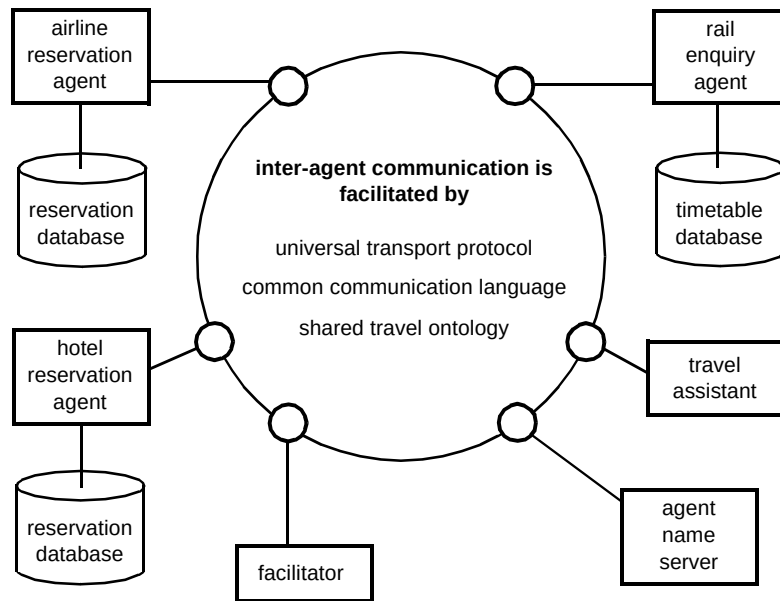


Fig 2 An agent-oriented solution to the personal travel assistant scenario.

4. The personal travel assistant application

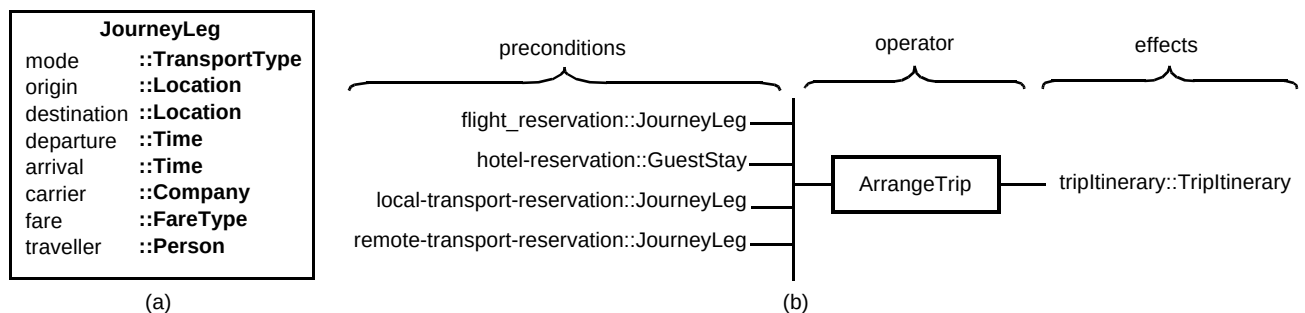
The scenario depicted in Fig 2 has been implemented using the ZEUS collaborative agent-building tool-kit, although this implementation did not address the issues of personalisation or post-plan monitoring.

The ZEUS tool-kit provides name server and facilitator agents for information discovery, and a support environment for users to specify (in a high-level manner) their own application-specific task agents. The specifications are automatically compiled to generate Java source code for the agents. ZEUS (user-defined or utility) agents communicate using the KQML agent communication language. The tool-kit also provides an ontology editing facility for users to define custom application-specific ontologies. The ontology editor supports a frame-based representation of concepts, i.e. in terms of their attributes and the values of these attributes. Figure 3(a) depicts the *JourneyLeg* ontology item, the key ontology concept used in this custom ontology for the travel domain. The *JourneyLeg* concept was used to

describe main unbroken segments of a journey, in terms of attributes such as origin and destination locations, mode of travel, the carrier, etc.

With respect to legacy software integration, ZEUS provides basic mechanisms for connecting user-defined agents to legacy applications, essentially exploiting facilities in the Java programming language. This enables the local database of an agent to be extended to encompass an external proprietary database. However, the user is expected to define the translation from the representation used in the external program into the ontology defined for the domain.

Reasoning and co-ordination are handled in ZEUS agents through a combination of planning, organisational structuring, contracting and negotiation. Figure 3(b) is a diagrammatic rendition of one of the plan operators used by the travel assistant agent in this implementation. The operator encodes knowledge about arranging transatlantic trips, e.g. to produce a *TripItinerary* (the effects of the operator), there must be a *flight-reservation*, a *hotel-reservation*, a *local-transport-*



74 Fig 3 (a) The *JourneyLeg* ontology item — its constituent attributes and their types, (b) the inputs and output of the *ArrangeTrip* plan operator.

reservation and a remote-transport-reservation (the preconditions of the operator).

Users can enforce restrictions on the ordering of the preconditions of operators, e.g. the constraint that the hotel and flight reservations should be achieved first, before the agent considers local and remote transportation. In addition, users can set constraints on the attributes of the preconditions or effects of operators, e.g. the two constraints:

`local-transport-reservation.destination`
`= flight-reservation.origin,`

`local-transport-reservation.arrival + 2`
`< flight-reservation.departure,`

state that the destination of the local transportation should be the location of origin of the flight (i.e. the origin airport), and further that the local transportation should arrive at that airport at most 2 hours before the departure time of the flight. Each plan operator also has an associated duration and cost, which are the expected time and cost required to perform the task defined by the operator. These costs may be arithmetic expressions with the preconditions and effects of the operators as parameters of the expression. Plan operators are only logical specifications of a task, and in ZEUS agents, external programs linked to the agent perform the actual task. The role of the agent is to determine when a task is applicable, while the external program executes the task. (This task execution may be something like printing a flight ticket and dispatching it to the owner.)

The travel scenario implementation consists of the agents shown in Fig 4 — a travel manager agent, a number

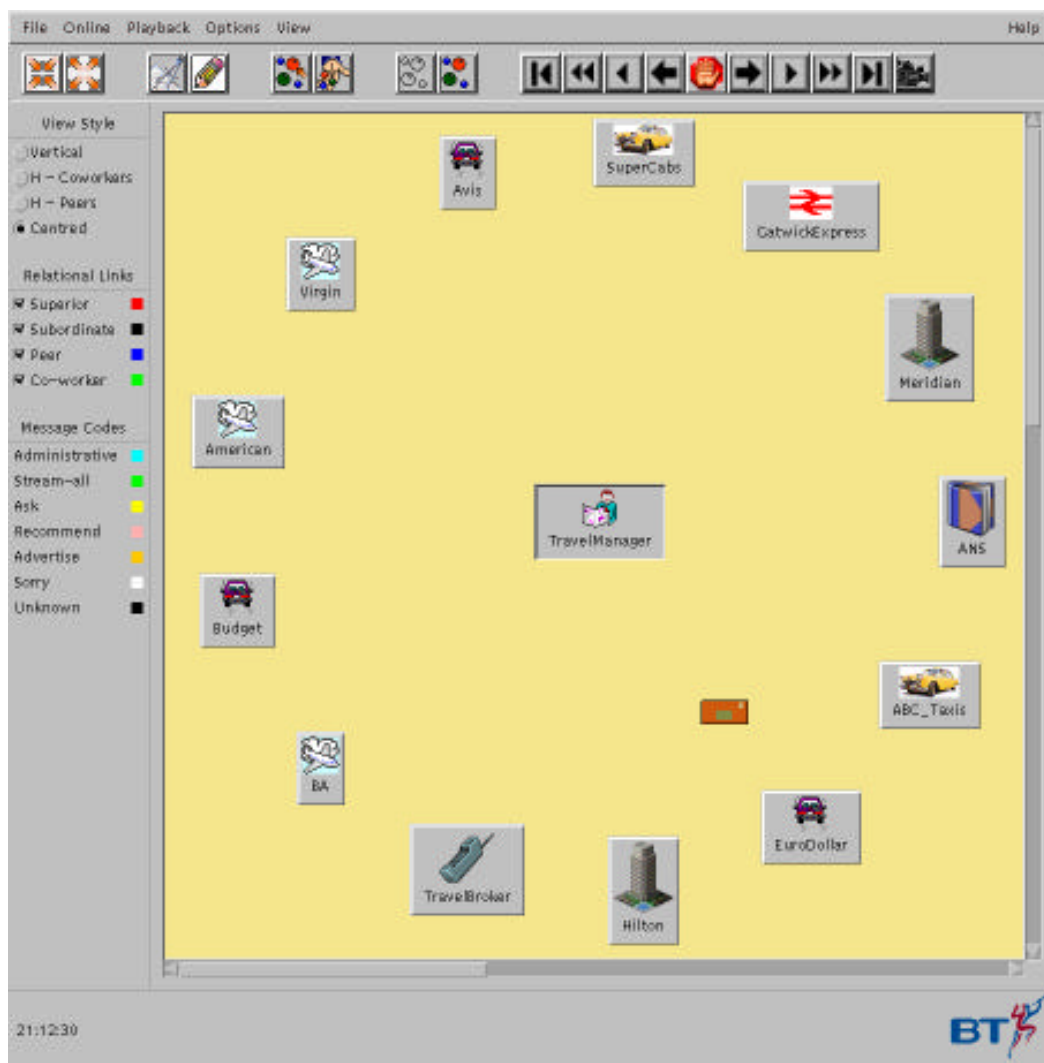


Fig 4 A view of the society of agents involved in the integrated travel assistance demonstrator. The envelope indicates a message being sent from one agent to another. The name server and facilitator agents are denoted ANS and TravelBroker respectively, and the travel assistant agent is denoted TravelManager (and shown at the centre).

of flight and hotel reservation agents, some local transportation agents, and the name server and facilitator agents. The travel manager was given the `ArrangeTrip` plan operator (as in Fig 3(b)), which describes how to arrange transatlantic trips. However, the travel manager can not perform the different tasks associated with reserving hotels, flights or local and remote transport. This knowledge is encoded in plan operators in the different reservation agents, with the agents managing their respective reservation databases.

Given the goal to create a `TripItinerary` for a transatlantic trip (with suitable inputs such as origin and destination locations, departure and return dates, etc), the travel manager selects the `ArrangeTrip` operator for execution as that operator has `TripItinerary` as one of its effects. However, the preconditions of the operator dictate that the travel manager should first get flight and hotel reservations, followed by reservations of local and remote transportation. As the travel manager has no operators capable of performing these activities, it must try to contract out the services.

To do so, the travel manager will first contact the facilitator agent to discover those agents in the society that can perform the required services. Next, it contacts the agent name server to determine the actual network addresses of the list of agents returned by the facilitator. Now, using the contract net co-ordination protocol, the travel manager announces the various reservation contracts to the reservation agents, receives bids from interested agents, and awards the contracts to the winning agents. The reservation agents bid for contracts depending on whether or not they can perform the task (e.g. a hotel reservation agent responds according to whether free rooms are available). In making a bid an agent will estimate the expected cost of performing the task, using their evaluation of the cost function of the relevant plan operator.

The travel manager can present the bids and let the user select the winning bid; the default behaviour provides a more convenient alternative, with the travel manager selecting the lowest bid as the winner of a contract. Because of the ordering constraints on the preconditions of the `ArrangeTrip` operator, the contracting process proceeds in stages. That is, the flight and hotel reservations are contracted out first. Next, the details of these contracts (e.g. source and destination airport locations, hotel location) are used to constrain the attributes of the local and remote transportation contracts. Finally, the remaining two contracts are placed. Figure 5 illustrates the society-wide plan created by the agents in an attempt to achieve the `TripItinerary` goal.

In this implementation, the travel manager agent is integrated with an electronic mail system and a computer telephony system; thus, once a trip has been arranged, it can notify the user via electronic mail or the telephone.

While this paper has focused on a personal travel assistance application, the problems described are typical of those facing developers of distributed applications where many disparate services must inter-operate. Agent technology offers practical solutions to these problems, enabling separate software systems to communicate unambiguously. Once systems can communicate, interaction becomes possible, raising the prospect of even more sophisticated behaviour, where the whole (the integrated service) is much greater than the sum of its constituent parts.

As the PTA demonstrator shows, the agents created from the ZEUS tool-kit have the ability to communicate, plan, and co-ordinate their activities. The ZEUS tool-kit has also been applied to other application areas, such as network management, home shopping and electronic commerce, areas where there is considerable scope for autonomous agents that act according to their local objectives, or in response to external events.

5. Related other work

There is little related work using agents to provide integrated travel assistance based on combining services from disparate sources. However, as was shown in section 3, there is a large body of work on agent-based approaches to problems in domains with similar characteristics to this travel scenario. Most work in the travel domain centres around the provision of automated reservation assistants that interactively learn traveller's preferences in a problem-solving process aimed at suggesting a service to the user that suits their preferences (see, for example, Linden et al [22]). A notable exception to this is the project reported by Völksen et al [23].

Völksen et al describe the specification of the German MOTIV (Mobility and Transport in Intermodal Traffic) program that aims to use collaborative agent technology to provide city-wide personal travel assistance, particularly trip planning and trip execution monitoring. The program plans to integrate many distributed travel-oriented services such as hotel and flight reservation systems, restaurant information systems, motorway and local traffic information systems, and parking management systems. In addition, it aims to exploit a number of different communication media and end-user devices. As the project is still in its infancy, no real evaluation is possible at this juncture. However, the project represents one of the largest-scale planned applications of agent technology. The sheer scale of the project indicates that, like ourselves, Völksen and his colleagues are confident of the potential of agent technology to provide solutions to real industrial problems.

6. Conclusions and further work

This paper has presented some of the problems inherent in providing integrated travel services to users through a medium such as the Internet. It has presented a collaborative

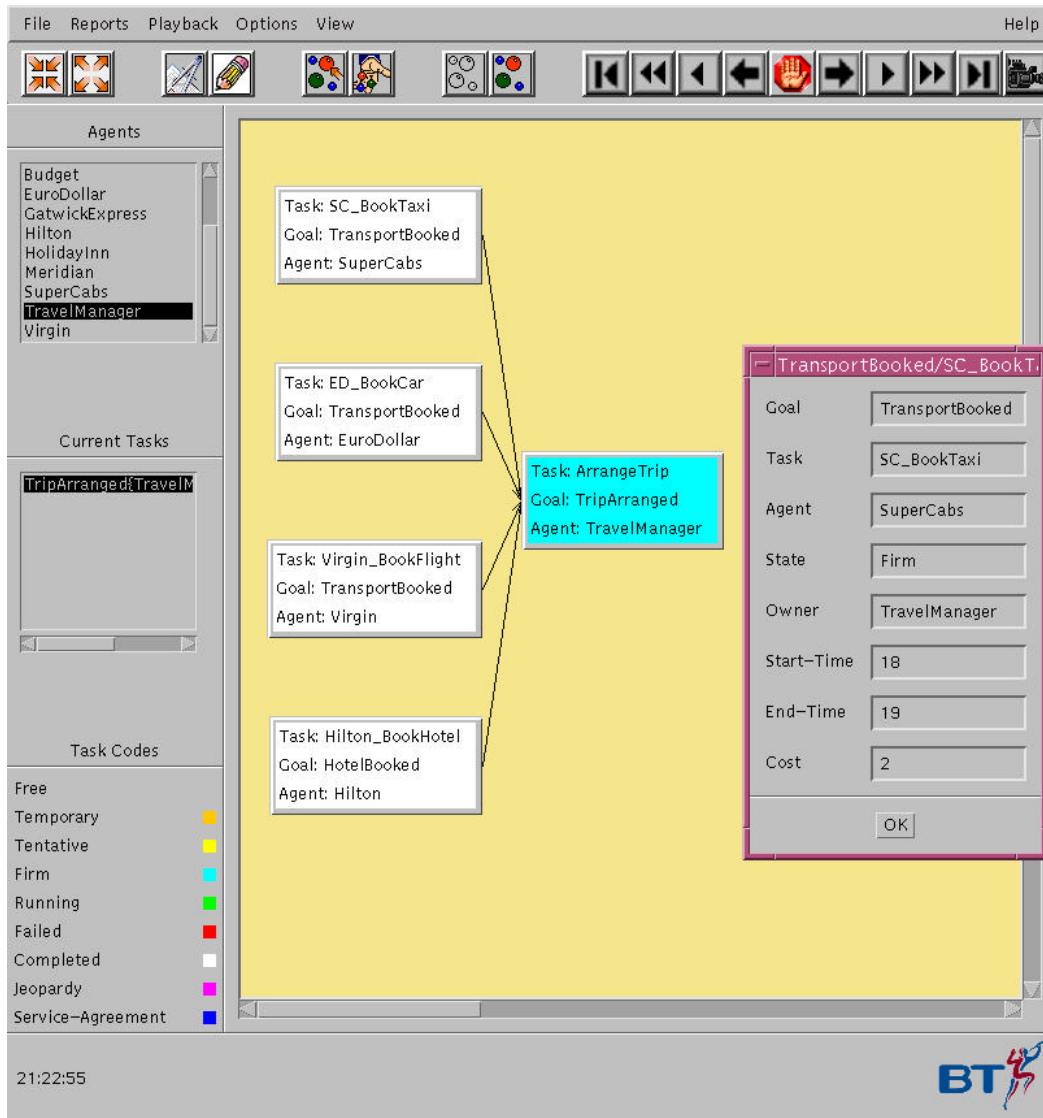


Fig 5 A view of the plan created by the travel manager agent in attempting to create the trip itinerary. As the plan indicates, the travel manager contracts in the services of four agents: two local transportation agents, EuroDollar and SuperCabs; a flight reservation agent, Virgin; and a hotel reservation agent, Hilton. The smaller window on the right shows details pertaining to one of the activities in the plan.

agent-based solution to the problem, and described an agent-based travel assistance demonstrator developed using the ZEUS collaborative agent-building tool-kit. In future work, it is intended to extend the initial demonstrator application to include personalisation of trip plans and post-plan trip monitoring.

Also implicit in this paper is the message that agent technology is now reasonably mature, and ready to start addressing real industrial problems.

References

- 1 Soltysiak S and Crabtree B: 'Automatic learning of user profiles — towards the personalisation of agent service', *BT Technol J*, **16**, No 3, pp 110—117 (July 1998).
- 2 Collis J C, Ndumu D T, Nwana H S and Lee L C: 'The ZEUS agent building tool-kit', *BT Technol J*, **16**, No 3, pp 60—68 (July 1998).
- 3 Kushmerick N: 'Wrapper induction for information extraction', PhD Thesis Technical Report No UW-CSE-97-11-04, Department of Computer Science and Engineering, University of Washington (1997).
- 4 'Catalogs for the Digital Marketplace', Research Report No 97-03, CommerceNet, available at <http://www.commerce.net/research/research.html> (1997).
- 5 Nwana H S: 'Software agents: an overview', *The Knowledge Engineering Review*, **11**, No 3, pp 205—244 (1996).
- 6 Nwana H S and Ndumu D T: 'An introduction to agent technology', *BT Technol J*, **14**, No 4, pp 55—67 (1996).
- 7 Jennings N R and Wooldridge M: 'Software agents', *IEE Review*, pp 17—20 (January 1996).

- 8 Judge D, Sheperdson J, Odgers B and Cui Z: 'Agent-enhanced workflow', *BT Technol J*, 16, No 3, pp 79—85 (July 1998).
- 9 O'Brien P D and Wiegand M E: 'Agents of change in business process management', *BT Technol J*, 14, No 4, pp 133—140 (1996).
- 10 Davison R, Hardwicke J and Cox M: 'Applying the agent paradigm to network management', *BT Technol J*, 16, No 3, pp 86—93 (July 1998).
- 11 Rao A S and Georgeff M P: 'BDI agents: from theory to practice', *Proc 1st International Conference on Multi Agent Systems*, San Francisco, California, pp 312—319 (1995).
- 12 Gallimore R J, Jennings N R, Lamba H S, Mason C L and Orenstein B J: '3D-scientific data interpretation using cooperating agents', in Nwana H S and Ndumu D T (Eds): 'Intelligent agents and multi-agent technology', *Proc 3rd Int Conf Practical Appl*, pp 47—56 (March 1998).
- 13 O'Brien P D and Nicol R C: 'FIPA — towards a standard for software agents', *BT Technol J*, 16, 3, pp 51—59 (July 1998).
- 14 Finin T and Labrou Y: 'KQML as an agent communication language', in Bradshaw J M (Ed): 'Software agents', MIT Press, Cambridge, MA, pp 291—316 (1997).
- 15 Searle J R: 'Speech acts', Cambridge University Press, Cambridge MA (1996).
- 16 Genesereth M R and Fikes R E (Eds): 'Knowledge interchange format, Version 3.0', Reference Manual, Computer Science Department, Stanford University, available at <http://www-ksl.stanford.edu/knowledge-sharing/papers/README.html#kif>
- 17 Lenat D B: 'CYC: a large-scale investment in knowledge infrastructure', *Communications of the ACM*, 38, No 11, pp 33—38 (1995).
- 18 Gruber T: 'A translation approach to portable ontology specifications', *Knowledge Acquisition*, 5, No 2, pp 199—220 (1993).
- 19 Genesereth M R and Ketchpel S P: 'Software agents', *Communications of the ACM*, 37, No 7, pp 48—53 (1994).
- 20 Nwana H S, Lee L and Jennings N R: 'Co-ordination in software agent systems', *BT Technol J*, 14, No 4, pp 79—88 (1996).
- 21 Davis R and Smith R G: 'Negotiation as a metaphor for distributed problem solving', *Artificial Intelligence* 20, pp 63—109 (1983).
- 22 Linden G, Hanks S and Lesh N: 'Interactive assessment of user preference models: the automated travel assistant', *Proc 6th Int Conf User Modelling*, pp 67—87 (1997).
- 23 Völksen G, Dieterich H and Steiner D: 'Intelligent agents for personal travel assistance', *Proc ITS Congress '97*, Berlin (1997).



Divine Ndumu graduated in Civil Engineering from Imperial College in 1989, and Mathematics and Computing from the Open University in 1991. In 1989, he was winner of the Imperial College Governor's prize for excellence in Civil Engineering. In 1993 he was awarded a PhD in Engineering Applications of Artificial Intelligence from Imperial College.

In 1992, he joined the University of Central Lancashire as a Lecturer, and in 1996/97 served a one year Research Fellowship secondment at BT Laboratories (BTL). In April 1997, Divine joined the Intelligent Systems Research Group at BTL, and currently leads the group's software agent research programme. In 1997, the ZEUS agent building tool-kit that he co-developed won the prestigious BCS IT award.

The focus of his current research is on the practical applications of AI techniques to real-world problems.



Jaron Collis obtained his BSc degree in Computation from the University of Manchester Institute of Science and Technology (UMIST), in 1993. In 1996 he completed his PhD at Queen's University, Belfast on the application of artificial intelligence to automatic scientific problem solving. He joined the intelligent systems research group at BT Laboratories in June 1997, and is currently working in the field of collaborative agents.



Hyacinth S Nwana is a principal research scientist/engineer and technical group leader in the Applied Research and Technology (ART) department at BT Laboratories. He holds a BSc in Computer Science and Electronic Engineering, an MSc in Computer Science and a PhD in Artificial Intelligence/Computer Science (1988). He also recently completed (November 1997) an MEd degree in Computer Science Education at Queen's College, the University of Cambridge.

Between 1989 and 1995, he worked in various roles (lecturer, researcher, visiting or contract scientist) for the Universities of Liverpool, Keele and Calgary (sponsored by the Royal Society), Shell Research Labs, Unilever Research Labs and BT Laboratories (BTL). He joined BTL fully in October 1995.

In 1991, he principally won the DEC European AI prize. In 1997, he led BT's agents-based project (ABW-ZEUS) which won the prestigious British Computer Society top award for innovation. He is a member of the British Computer Society and a Chartered Engineer.

He currently runs ART's Future Technologies group investigating novel biologically motivated computing models, software agents, believable interface agents, cognitive systems, and the application of such techniques to telecommunications and other computing problems.